实验三：MPI 分布式图神经网络推理实验报告

姓名：戴铭秋 学号：2024201526

一、实验目的与主要结论

本实验在四台华为云鲲鹏 ECS 上实现分布式图神经网络前向推理，比较集合通信 halo 交换、点对点通信与计算重叠，以及全量 Allgather 基线，分析进程分布、图划分、计算通信比和负载均衡对性能的影响。

默认图上，四机四进程 mpi2 的中位时间为 **0.4839 秒**，相对独立串行参考 **1.655455 秒**，加速比为 **3.42**、效率为 **0.86**。四机八进程的 mpi2 降至 **0.3695 秒**，但波动更大。单机四进程在两核机器上出现负优化。强社区结构适合 block 划分；高阶节点集中时，cyclic 可通过均衡负载抵消额外通信。

二、实验环境与计算模型

项目	实际配置
集群	ecs-hw-0001 ~ ecs-hw-0004，共四台，使用私网通信
系统	openEuler 20.03 LTS，aarch64
处理器	每台 2 vCPU，HiSilicon，lscpu 显示 2.9 GHz
内存	每台系统可见内存 2844 MiB
MPI	MPICH 3.3.2，统一路径 /usr/local/mpich
编译环境	GCC 7.3.0；-O3 -std=c++17 -Wall
图生成环境	Python 3.7.9、NumPy 1.21.6
默认图	200000 节点，3999004 条有向边，输入特征 64 维
默认生成参数	degree=20、communities=8、intra=0.70、seed=2025
默认网络	2 层，隐层 64 维；权重由框架固定种子生成

每层计算为：

$$H^{(l+1)} = \sigma\Big((D^{-1}AH^{(l)})W^{(l)}\Big).$$

即先对邻居特征求平均，再进行线性变换，除最后一层外使用 ReLU。各进程负责自己的节点，计算所需但属于其他进程的邻居称为 halo 节点。多条跨进程边可能引用同一个远程节点，因此 halo 必须去重；每层表示都会变化，L 层推理需要交换 L 次。

三、实现方法

3.1 通信计划建立

`gnnSetup` 扫描局部 CSR，将远程邻居按 owner 分组、排序并去重。通过 `MPI_Alltoall` 交换请求数，再用 `MPI_Alltoallv` 交换请求的全局节点编号。发送方把请求编号转成本地索引，形成发送表；接收方按来源进程连续存放 halo，随后调用 `remapLocalIndices`。计划在推理计时前建立一次，各层沿用同一节点需求关系，但传输当前层的表示。

本地节点进一步重排为内部节点前缀和边界节点后缀，同时调整 CSR 行、特征、全局编号与映射。每行邻居顺序保持原样，以保留原计算核的累加顺序。MPI 的 counts/displs 以元素为单位，各层均将节点数乘当层输入维度。

3.2 三种通信策略

策略	执行过程
mpi1：集合通信	打包需求节点 → MPI_Alltoallv 仅交换 halo → 原有 SpMM 核
mpi2：点对点与重叠	Irecv → Isend → 计算内部节点 → Waitall → 计算边界节点
allgather：全量基线	Allgatherv 收集全部表示 → 映射到本地/halo 缓冲区 → SpMM

mpi2 在等待通信前执行不依赖 halo 的内部节点计算；等待完成前发送缓冲区保持有效、接收中的 halo 不被读取。Allgather 显式保存 rank 顺序到全局编号的映射，因此支持 cyclic 划分及本地节点重排；其全量缓冲区仍随整图规模增长。原有计算核、串行参考和校验逻辑未修改。

四、正确性与测量方法

4.1 正确性检查

使用原 `checkResult` 收集结果，与串行参考逐元素比较，并检查每个节点恰好输出一次，容差为 `atol=rtol=1e-7`。

覆盖情况	配置及结果
单/多进程、两种划分、单/多层	P=1/4、block/cyclic、L=1/3，共 8 组
节点数不能整除进程数、孤立点	N=203、d=7、hidden=11，198 条有向边、77 个孤立点
空分区	N=3、0 条边、P=4、L=3，两种划分各一组
检查总计	10 组边界配置 × 3 策略，30 项全部通过
性能配置检查	19 组配置 × 3 策略，57 项全部通过

四台可执行文件的 SHA-256 一致。每份图生成后分发到四台的相同路径，并核对 SHA-256；证据保存在 环境日志 和 图数据校验记录。

4.2 计时与加速比

每组预热 1 次、正式重复 10 次。每次计时前调用 `MPI_Barrier`，使用 `MPI_Wtime`，并取所有 rank 耗时的最大值作为墙钟时间。计时包含缓冲区分配、打包、通信、等待和计算，排除图加载、划分、建计划、串行参考、校验和打印。报告给出中位数，完整最小值和最大值保存在附表。

框架在 rank 0 计算串行参考时，其他 rank 的忙等可能争用 CPU，尤其影响单机四进程。因此另用独立驱动调用同一个 `gnnSerial`，预热 1 次、重复 10 次，取中位数；同图、层数、维度的各并行配置统一使用这一基准。不同图或网络参数分别测量对应的串行基准，原框架单次计时仍完整保留。

$$S(P) = \frac{T_{serial}}{T_{parallel}(P)}, \qquad E(P) = \frac{S(P)}{P}.$$

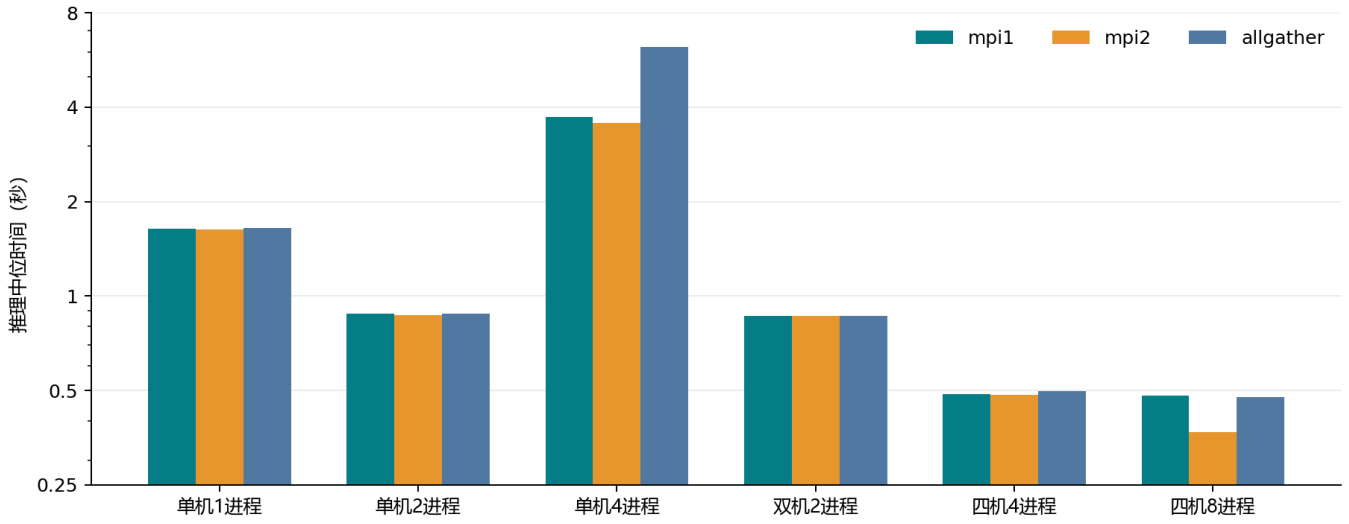
默认配置的独立串行基准为 **1.655455232 秒**。

五、性能结果与分析

下列推理时间均为秒，均取 10 次正式测量的中位数。

5.1 进程数与进程分布

进程分布	mpi1	mpi2	allgather	mpi2 最小/最大	mpi2 加速比	mpi2 效率
单机 1 进程	1.6362	1.6341	1.6448	1.6299 / 1.6363	1.01	1.01
单机 2 进程	0.8778	0.8696	0.8811	0.8624 / 0.8758	1.90	0.95
单机 4 进程	3.7129	3.5591	6.2371	1.9105 / 3.7322	0.47	0.12
双机 2 进程	0.8652	0.8640	0.8631	0.8574 / 0.9701	1.92	0.96
四机 4 进程	0.4871	0.4839	0.4973	0.4706 / 0.5168	3.42	0.86
四机 8 进程	0.4828	0.3695	0.4758	0.3362 / 0.7335	4.48	0.56



单机四进程争用两核，MPI 忙等和调度竞争使耗时明显上升。检查时各进程 RSS 合计约 1 GiB，未使用交换空间，未观察到内存耗尽。四机八进程用满 8 核，但 halo 总数由 466026 增至 739026，扩展受到通信与调度开销限制；mpi2 的最小/最大时间为 0.3362/0.7335 秒，波动不可忽略。

5.2 图结构与划分方式

intra	划分	跨进程边比例	halo 总数	内部节点比例	mpi1	mpi2	allgather
0.10	block	67.6%	593283	0.00%	0.5558	0.5566	0.5319
0.10	cyclic	75.0%	595955	0.00%	0.5620	0.5622	0.5442
0.70	block	22.5%	466026	1.12%	0.4871	0.4839	0.4973
0.70	cyclic	75.0%	595926	0.00%	0.5168	0.5098	0.5024
0.95	block	3.7%	132231	47.43%	0.4151	0.4180	0.4585
0.95	cyclic	75.0%	595947	0.00%	0.4879	0.4795	0.4796

社区结构强时，block 保留局部性，减少跨分区依赖；cyclic 会打散社区。intra=0.95、block 时，两层推理所有 rank 合计的 halo 逻辑接收量约 64.6 MiB，全量基线约 293.0 MiB，二者均不代表实测物理链路字节数。局部性弱时 halo 接近全部远程节点，打包和不规整交换的优势减小，Allgather 可能更快。

默认四机四进程只有约 1.12% 内部节点，可重叠工作很少；intra=0.95、block 时内部节点增至 47.43%，但 mpi2 仍略慢于 mpi1。非阻塞接口和重叠路径不保证一定加速，MPI 进展、计算占比及额外开销均可能影响结果；本实验未分阶段采集，不能进一步定量归因。

5.3 层数与隐层维度

网络参数	mpi1	mpi2	allgather	划分 + 建计划
L=1，hidden=64	0.2285	0.2265	0.2387	0.0279
L=2，hidden=32	0.2753	0.2763	0.2831	0.0314
L=2，hidden=64	0.4871	0.4839	0.4973	0.0296
L=2，hidden=128	1.1868	1.1802	1.1811	0.0296
L=4，hidden=64	0.9922	1.0024	0.9912	0.0278

增加层数会增加计算量与通信次数；提高 hidden 还会增大线性变换的计算量。因此 hidden=128 的耗时增幅不能仅由通信字节数解释。不同网络规模属于不同任务，加速比均使用各自对应的串行参考。

5.4 负载均衡

固定 `skew=0.02`、`skew-degree=400`，改变高阶节点的编号分布范围。

hub-span	划分	边负载 max/mean	halo 总数	mpi2	allgather
0.1	block	1.668	502276	0.7992	0.7838
0.1	cyclic	1.006	599477	0.6301	0.6372
1.0	block	1.007	581876	0.6277	0.6086

hub-span	划分	边负载 max/mean	halo 总数	mpi2	allgather
1.0	cyclic	1.008	599481	0.6337	0.6189

高阶节点集中在前 10% 编号区间时，block 的边负载最大值/平均值达到 1.668，cyclic 降至 1.006，mpi2 从 0.7992 秒降为 0.6301 秒。尽管 cyclic 的 halo 更多，负载均衡仍带来收益。高阶节点均匀散布时，两种划分的负载接近，cyclic 的优势消失。边数仅是工作量代理，线性变换还按节点付出成本，不能将边负载比例直接等同于耗时比例。

六、Linux 性能工具分析

`perf list` 在当前虚拟机上仅列出软件事件，`cycles`、`instructions` 实测为 `<not supported>`。使用 `task-clock`、`context-switches`、`page-faults`，并用 `/usr/bin/time -v` 补充资源统计。MPI 将采集器放在 rank 0 的可执行命令外，其余 rank 正常执行，避免把 mpirun 和所有进程混为一个采集对象。

串行驱动直接调用原 `gnnSerial`；MPI 使用 `--only <策略> --no-check --warmup 1 --repeats 10`，避免串行参考及校验污染计数。两者均执行 11 次推理；计数仍包含进程启动、图加载，以及 MPI 的划分和建计划，不能当作单次纯推理计数。

对象	task-clock / ms	自愿/非自愿切换	缺页次数	峰值 RSS / MiB
串行	18,242.46	6/32	27,095	223.2
mpi1 / rank 0	5,369.23	26/61	10,605	205.9
mpi2 / rank 0	5,535.86	25/75	10,606	205.9
allgather / rank 0	5,461.02	26/38	20,236	227.0

Allgather 的 rank 0 峰值 RSS 高于两种 halo 策略，缺页次数约为其 1.9 倍，与额外全量表示缓冲区的使用相符。task-clock 包含 MPI 忙等；缺页以次缺页为主，time 记录 major faults=0、swaps=0，不能据此推断缓存命中率或磁盘交换。当前权限下 perf 自动使用 `:u`，`context-switches:u` 均为 0，不代表没有调度切换；表中切换数取自 time。RSS 为采集命令树内的最大常驻集，MPI 数据仅代表 rank 0，不能当作集群总内存。

七、结论、局限与改进方向

实验说明，MPI 扩展收益同时受到 CPU 资源、halo 规模和负载均衡制约。单机超配进程数是本次明确观察到的负优化；强社区图适合 block，而高阶节点集中时 cyclic 可改善负载。通信与计算重叠需要足够的内部工作，并依赖实际通信进展，不能仅凭使用非阻塞接口判断效果。

本实验按固定策略顺序运行，云主机噪声与资源共享可能影响小幅差异，未进行显著性检验。框架每个 rank 仍完整加载全图 CSR 和初始特征，优化的是层间表示通信，尚未实现整图存储分片。硬件计数器不可用，且只采集 rank 0，结论不能直接外推到训练、GPU 或更大集群。

后续可采用更好的图划分，仅保存本地 CSR/特征，按维度复用缓冲区，并在明确 CPU 预算下评估异步进展线程。

附录：复现方法与完整数据

云端工作目录为 `/home/dmq/lab3-20261114`。默认图生成及单次运行命令：

```
export PATH=/usr/local/mpich/bin:$PATH
export LD_LIBRARY_PATH=/usr/local/mpich/lib${LD_LIBRARY_PATH:+:$LD_LIBRARY_PATH}
cd /home/dmq/lab3-20261114
make
python3 gen_graph.py --nodes 200000 --degree 20 --feat-dim 64 \
  --communities 8 --intra 0.70 --seed 2025 --out graph.bin
# 运行前保证四台机器的 main 与 graph.bin 位于相同路径且内容一致
mpirun -f hostfile_spread -np 4 ./main --graph graph.bin \
  --part block --layers 2 --hidden 64 --warmup 1 --repeats 10
```

全部配置由 `python3 run_experiments.py` 执行，包含图生成、分发、校验和检查；`bash run_profiles.sh` 在主实验结束后执行独立串行基准及性能采集。`hostfile_single` 为单机 4 slots，`hostfile_spread` 为每机 1 slot，`hostfile_spread8` 为每机 2 slots。

- 完整数据表：57 项策略结果的中位数、最小值、最大值、加速比及效率。
- CSV 数据：包含建计划耗时、halo、负载及串行基准等字段。
- 逐步实验记录。
- 主实验原始日志与性能采集完成记录。